



Aror University of Art, Architecture, Design & Heritage Sukkur.

Department of Artificial Intelligence, Multimedia Gaming and Cyber Security

Object Oriented Programming (Fall-2025)

LAB No. 5

Objective of Lab No. 5:

After performing lab3, students will be able to:

- Implement classes in Java and Create Objects
- Use Default constructor and new operator
- Use Dot Operator
- Modify default and create Parameterized constructor
- Implement void and return type methods

Recap of Last Lab -4:

1. **Create a java class and do the following:**
 - a. Create an array to store 4 seasons of an year
 - b. Use foreach loop to print each of the array element
 - c. Use for loop to iterate over the array elements, and inside the loop use if-else-if statement to check the season and print months of that particular season.
2. **Generate the following sequences using both for loop and while loop separately.**

run:

1	39
4	34
7	29
10	24
13	19
16	14
19	9
22	4

3. **Create a matrix like shown below:**

12	13	15	16
11	110	121	17
17	18	100	21



Aror University of Art, Architecture, Design & Heritage Sukkur.

- a. Divide each even number from the matrix by 2 and store the updated value in the matrix.
 - b. Now use for-each loop to iterate over the matrix and display the Odd Numbers from the matrix.
 - c. Use for loop to do the sum of updated even numbers from the matrix
4. **Write a java program to do the following:**
 - a. Populate an array having size 10 using a do while loop with following sequence: 1, 4, 9, 16, 25, 36, 49, 64, 81, 0 using a do while loop.
 - b. Now use while loop to do the sum of odd numbers from this array and skip the even numbers using appropriate jump statements.
 - c. Break the loop as soon as 81 is encountered.
5. **Design a simple seat reservation system for a theatre, the Algorithm for the program is given below:**
 - a. Create theatre structure by determining the number of rows and number of columns, rows and columns will determine the total number of seats.
 - b. Use a do while loop to display the following messages, loop will be terminated when user presses 3:
 - i. Display available seats: 1
 - ii. Reserve a seat: 2
 - iii. Exit: 3
 - iv. Enter your choice:
 - c. Now use a switch case statement inside the do while loop
 - i. If user presses 1 then display the available seats, available seats are represented by false and unavailable seats are represented by true.
 - ii. If user presses two, then ask the user row number and column number (Hint: subtract -1, indexing starts from zero)
 - iii. Check if the seat is not reserved already then reserve the seat, otherwise display seat is already reserved.
 - iv. If row number and column number are out of the range then display invalid range.
 - d. Default choice of switch will display invalid choice, if the user provides any number which is out of the given range (1-3)
6. **Write java program to check whether a string is Palindrome or not (Hint: use `string.charAt()` method)**



Aror University of Art, Architecture, Design & Heritage Sukkur.

Lab 5: Class

Introduction to Class:

In object-oriented programming (OOP), a class is a blueprint or template used to create objects. It defines what an object is and what it can do by specifying its data (variables) and behavior (methods).

Definition: A class is a user-defined data type that groups data members (variables) and member functions (methods) into a single unit.

Use Classes in Programming:

Classes help us to:

- Organize large programs into logical parts
- Represent real-world entities (e.g., Student, Car, Employee)
- Write reusable and manageable code
- Improve readability and maintenance

Without classes, programs become long, messy, and hard to understand.

Classes in Java:

Java is a pure object-oriented language, which means:

- Everything is written inside a class
- Programs cannot exist without a class
- The `main()` method itself is placed inside a class

What Does a Class Contain?

A class can contain:

- Variables → represent data
- Methods → represent actions
- Constructors → initialize objects



Aror University of Art, Architecture, Design & Heritage Sukkur.

Use Cases:

Classes are used when:

- Creating real-world models (Student, Teacher, BankAccount)
- Managing records and data
- Building large applications
- Implementing OOP concepts (inheritance, encapsulation, etc.)

Lab 5: Tasks

Task-1: Create a **Java class** named `Student`. Declare two instance variables:

- `int id`
- `String name`

In the `main()` method: Create one object of the `Student` class. Use the dot operator to assign values to `id` and `name`. Print the values on the console.

Solution:

```
public class Student {  
    int id;  
    String name;  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.id = 101;  
        s1.name = "Ali";  
        System.out.println("Student ID: " + s1.id);  
        System.out.println("Student Name: " + s1.name);  
    }  
}
```



Aror University of Art, Architecture, Design & Heritage Sukkur.

Task-2: Create a class named `Circle` that has: One attribute: `radius`. In the `main` method: Create one object of the `Circle` class using the default constructor. Use the dot operator to:

- Assign a value to the `radius` of the object.
- Print the value of the `radius` on the console.

Solution:

```
class Circle {  
    double radius;  
    public static void main(String[] args) {  
        Circle c1 = new Circle();  
        c1.radius = 7.5;  
        System.out.println("Radius of the circle: " + c1.radius);  
    }  
}
```

Task-3: Create a Java class named `Student`, Declare the following instance variables then

- `int id`
- `String name`
- `int marks`

Create a method named `displayInfo()` which: Does not return any value, Does not take any parameters, Prints student id, name, and marks.

In the `main()` method: Create two objects of the `Student` class, Use the dot operator to initialize the instance variables of both objects, Call the `displayInfo()` method for each object.

Solution:

```
class Student {  
    int id;  
    String name;  
    int marks;  
    void displayInfo() {
```



Aror University of Art, Architecture, Design & Heritage Sukkur.

```
System.out.println("Student ID: " + id);
System.out.println("Student Name: " + name);
System.out.println("Marks: " + marks);
System.out.println("-----");    }
public static void main(String[] args) {
    Student s1 = new Student();
    s1.id = 101;
    s1.name = "Ali";
    s1.marks = 85;
    Student s2 = new Student();
    s2.id = 102;
    s2.name = "Sara";
    s2.marks = 92;
    s1.displayInfo();
    s2.displayInfo();
}
}
```

Task-4: Write a Java program, create a constructor which pass an integer and a string variables. The passing values assigned to the instance variables of class. Create another function to display the values of variables. In the main function

Solution:

```
class Student {
    int id;
    String name;
    // Constructor
    Student(int i, String n) {
        id = i;
        name = n;
    }
}
```



Aror University of Art, Architecture, Design & Heritage Sukkur.

```
void display() {  
    System.out.println("ID: " + id);  
    System.out.println("Name: " + name);  
}  
  
public static void main(String[] args) {  
  
    // Creating object and calling constructor  
    Student s1 = new Student(101, "Ali");  
  
    // Displaying data  
    s1.display();  
}  
}
```

Lab Exercises:

1. **Create a class circle with attributes radius and color, Now:**
 - a. create two objects namely red_circle and green_circle in the main method using default constructor.
 - b. Add a method named calculateArea() which calculates the area of calling object



Aror University of Art, Architecture, Design & Heritage Sukkur.

and prints it on the console screen, note the method does not return any value and accepts no parameter.

- c. Use dot operator to initialize the instance variables of red_circle and green_circle (Use any radius value)
 - d. Now print the radius of both circles (Use Dot Operator with instance variables)
2. **Now Modify the Task 1 in following way:**
- a. Initialize the instance variables using parameterized constructor (Demonstrate bypassing the local variable hiding)
 - b. calculateArea() method returns the area of calling object, choose return type wisely.
 - c. Print the radius of both circles with the help of getter methods for color and radius.
3. **Create a class called Bank Account with attributes balance and AccountNumber, Now:**
- a. Create two Objects namely acc1, acc2 with parameterized constructor, by bypassing local variable hiding (Initial balance of account 1 is 1000 and account 2 is 500).
 - b. Implement methods to deposit, withdraw and check balance of an account.
 - c. Deposit 500 in account 1 and 1500 in account 2 with the help of method.
 - d. Check balance of both accounts now.
 - e. Withdraw 500 from account 1 (Make sure you have that much balance...)
 - f. Check balance of account 1.
 - g. Withdraw 3000 from account 2.
4. Write a program that would print the information (name, year of joining, salary, address) of three employees by creating a class named 'Employee'. The output should be as follows:

Name	Year_of_Joining	Address
Robert	1994	WallsStreat
Sam	2000	WallsStreat
John	1999	WallsStreat

5. **Create a class student having attributes id, name and grades (array of 5 double elements)**
- a. Initialize the above given attributes with the help of parameterized constructor by creating 2 objects.
 - b. Add a method display_average_grade() which displays the average grade of calling object.
 - c. Add a method calc_percentage() which converts each numeric grade into percentage form and returns the array of percentages, remember naximum marks



Aror University of Art, Architecture, Design & Heritage Sukkur.

- for each grade are 200.
- d. Add a method called `concat_id_name`, which concatenates the id and name of student and returns the concatenated result.
 - e. Demonstrate the use of all the above methods by calling them over the objects.
6. **Create a class `matrix`, with attributes `number of rows` and `number of columns`, and a `matrix` of size `rows` and `columns` named `matx`:**
- a. Add a constructor which sets the number of rows and number of columns when an object is created.
 - b. Add a `get_matrix()` method which displays the elements of `matx` of a `matrix` object
 - c. Add a method called `set_element()` which accepts row number and column number along with the value, and sets the value at that row number and column number.
 - d. Create an object `matrix1` having 4 rows and 3 columns.
 - e. Create an object `matrix2` having 3 rows and 3 columns.
 - f. Initialize the `matx` attribute of both matrices.
 - g. Update the element of `matx` of `matrix1` at row number 1 and column number 2 with 3.
 - h. Call `get_matrix()` on both objects.